Apellidos: .....Nombre:.....Grupo:.....DNI:.....

Examen EP2 (temas 8-14)

Duración: 2 horas 30 minutos. La solución de cada ejercicio se tiene que escribir en el espacio reservado para ello en el propio enunciado. No podéis utilizar calculadora, apuntes, etc. La solución del examen se publicará en Atenea el 23 de diciembre por la tarde y las notas antes de las 23:59 del 2 de enero.

Ejercicio 1 (1.5 puntos) El programa en ensamblador SISA que se muestra a continuación se ha traducido a lenguaje máquina situando la sección de datos (.data) a partir de la dirección 0xC000 y justo a continuación la sección de código (.text).
Una vez ensamblado y cargado el programa en la memoria y antes de comenzar su ejecución:

```
.data
        N = 5
V1:     .word 0x0100, 0x0302, 0x0504
        .even
Res:    .space 2

.text
        MOVI    R1, lo(N)
        MOVHI   R1, hi(N)
        MOVI    R2, lo(V1)
        MOVHI   R2, hi(V1)
        MOVI    R3, lo(Sub)
        MOVHI   R3, hi(Sub)
        MOVI    R4, 0
        ADDI    R5, R2, 0
loop:   JALR    R7, R3
        LDB     R6,0(R2)
        ADD     R4, R6, R0
        ADDI    R2, R2, 1
        ADDI    R1, R1, -1
        BNZ     R1, loop
        ST      6(R5), R4
        …
sub:    IN      R0, Key_status
        BZ      R0, sub
        IN      R0, Key_data
        JALR    R7, R7
.end
```

a. ¿A qué direcciones de memoria corresponden las etiquetas o direcciones simbólicas siguientes?

Loop = Res =

b. ¿Cuál es la dirección de memoria y su contenido donde han quedado almacenadas las siguientes instrucciones una vez cargado el programa?

Instrucción	@memoria y contenido
`MOVHI R1, hi(N)`	Mem_w[0x]=0x
`BNZ R1, loop`	Mem_w[0x]=0x

c. Una vez ejecutado el programa en el SISC Von Neumann, asumiendo que el usuario teclea un "5" cada segundo hasta que finaliza el programa, ¿Cuál es la dirección de la memoria donde ha escrito la instrucción (ST 6(R5), R4) y cuál es su contenido?

Mem_w[0x]=0x

Ejercicio 2 (0,5 puntos) Indica el contenido de la ROM_Q+ en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado).

`ROM_Q+[0x00A]`	0x	`ROM_Q+[0x02A]`	0x
`ROM_Q+[0x22E]`	0x	`ROM_Q+[0x035]`	0x

Ejercicio 3 (1 punto) Completa la siguiente tabla ensamblando las instrucciones en ensamblador SISA o desensamblando las instrucciones en lenguaje máquina según sea necesario. Indica poniendo NA en la casilla aquellos casos en los que la instrucción no corresponda al lenguaje SISA.

Lenguaje máquina SISA (en hexadecimal)	**Lenguaje ensamblador SISA**
0x0000	
0x8F11	
	`SUBI R5, R5, 1`
	`OUT 16, R3`

Ejercicio 4 (2 puntos) Indica el contenido de la ROM_OUT en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado).

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Contenido ROM_OUT** (en hexadecimal)
`F`	`JALR R3, R5`	0x
`Addr`	`ST 0x10(R5), R3`	0x
`Out`	`OUT 0, R7`	0x
`Bnz`	`BNZ R2, 10`	0x

Ejercicio 5 (1 punto) Indicad qué cambios se producen en el estado del computador **SISC Von Neumann** después de ejecutar cada una de las instrucciones de la tabla suponiendo que antes de ejecutarse cada una de ellas el PC vale 0x2228, el contenido de todos los registros es 0x1234, el byte contenido en todas las direcciones pares de la memoria es 0x22 y el de todas las impares es 0x11 y el contenido del port PRINT_STATUS es 1. Utilizad la notación Ri=0x… para un cambio en el registro "i", MEMb[0x…]=0x... y/o MEMw[0x…]=0x... para indicar cualquier cambio en la memoria y PORTIN[0x...]=0x... para los puertos de entrada y PORTOUT[0x...]=0x... para los de salida.

Instrucción a ejecutar	**Cambios en el estado del computador**
`JALR R5, R5`	
`ST 6(R5), R3`	
`OUT PRINT_DATA, R3`	
`BNZ R7, 0xFF`	

Apellidos: ……Nombre:……Grupo:……DNI:……

Ejercicio 6 (1 punto) Completad las filas y columnas de la siguiente tabla que representa un subconjunto de la palabra de control del SISC Von Neumann. Indicad solo los valores necesarios para realizar las acciones correspondientes a cada estado. Para el resto de valores, ya sean desconocidos o **no sean necesarios para ninguna acción del estado,** poned x.

Nodo / Estado	**Instrucción en IR** (en ensamblador)	**@A**	**@B**	**@D**	**F**	**N** (en hexadecimal)	**ADDR-IO** (en hexadecimal)
F	`MOVHI R3, 0xFF`						
Addr	`LDB R3, -1(R0)`						
Movhi	`MOVHI R3, 16`						
Bnz	`BNZ R7, 4`						

Ejercicio 7 (1 punto) Especificad el camino crítico (indicando la suma ordenada de los tiempos de propagación de los bloques por los que pasa) y calculad el tiempo de ciclo mínimo para que el computador SISC Von Newmann pueda ejecutar correctamente los siguientes estados. No debéis añadir ningún porcentaje de seguridad al cálculo del tiempo de ciclo mínimo. Suponed que los tiempos de propagación de los bloques que forman el computador son los siguientes:

Tp**(ROM_Q+)** = 50 u.t.
Tp**(ROM_OUT)** = 60 u.t.
Tp**(MUX-2-1)** = 50 u.t.
Tp**(MUX-4-1)** = 100 u.t.
Tp**(REG)** = 150 u.t. // Tp de un registro.
Tp**(REGFILE)** = 250 u.t. // Tiempo de lectura/escritura de un registro.
Tp**(ALU-slow)** = 650 u.t. // Tp de ALU para operaciones lentas: ADD, SUB, CMP*.
Tp**(ALU-quick)** = 350 u.t. // Tp de ALU para operaciones rápidas: todas menos las anteriores.
Tacc**(64Kb MEMORY)** = 1000 u.t. // Tiempo de acceso (lectura o escritura) a la memoria. Fijaros que este tiempo incluye toda la lógica empleada para construir-la a partir de dos bloques de 32Kb.
Tp**(AND-2)** = Tp**(OR-2)** = 20 u.t.
Tp**(NOT)** = 10 u.t.

Desconocemos como se han implementado los bloques internamente (y podría ser diferente a como los hemos visto en clase). Recordad que un registro con señal de carga (Ld), REGwLd, está construido con un REG y un MUX-2-1 (no os facilitamos su esquema porque lo deberíais saber).

a. **D (Decode) =**

b. **Ld =**

Apellidos: ……Nombre:……Grupo:……DNI:……

Ejercicio 8 (2 puntos) Queremos empezar a adaptar nuestra arquitectura Von Newmann para que pueda trabajar con enteros de 32 bits. Para ello empezaremos añadiendo una instrucción que pone a cero 32 bits de memoria. La instrucción e la siguiente:

Sintaxis: SETZero-32 Ra **Codificación**: 1011 aaa x xxxx xxxx

Semántica: PC = PC+2; MEMw[Ra] = 0; MEMw[Ra+2] = 0

a. ¿Cómo modificaríais el **hardware de la UPG y el grafo de estados** para poder implementar esta nueva instrucción? Podéis modificar buses y añadir multiplexor(es), buses y señales de control, pero no podéis modificar ni los bloques ni el uso de las señales de control existentes. Describe las modificaciones con un dibujo de la parte relevante o mediante una explicación. Indicad también como cambiaria el grafo de estados.

b. Indicad las acciones que se deberán ejecutar en cada uno de los estados de la nueva instrucción (**Sin incluir Fetch y decode**). La tabla adjunta tiene el número suficiente de filas, incluso es posible que no sean todas necesarias.

Estado	Acciones

c. Indicad el contenido de las filas de la ROM_OUT que sea preciso modificar (**sin incluir Fetch y decode**). La tabla adjunta tiene el número suficiente de filas, incluso es posible que no sean todas necesarias. También hemos añadido un par de salidas nuevas por si las necesitas. De nuevo, puede que no las necesites todas.

@ROM	New1	New2	Bnz	Bz	WrMem	RdIn	WrOut	WrD	LdIr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/L/A1	P/I/L/A0	OP1	OP0	MxN1	MxN0	MxF	F2	F1	F0	Mx@D1	Mx@D0